

extractFeatures

Extract interest point descriptors

Syntax

```
[features,validPoints] = extractFeatures(I,points)
[features,validPoints] = extractFeatures(I,points,Name=Value)
```

Description

`[features,validPoints] = extractFeatures(I,points)` returns extracted feature vectors, also known as descriptors, and their corresponding locations, from a binary or intensity image. example

The function derives the descriptors from pixels surrounding an interest point. The pixels represent and match features specified by a single-point location. Each single-point specifies the center location of a neighborhood. The method you use for descriptor extraction depends on the class of the input points.

`[features,validPoints] = extractFeatures(I,points,Name=Value)` specifies options using one or more name-value arguments in addition to any combination of arguments from previous syntaxes. For example, `extractFeatures(I,points,Method="Block")` sets the method to extract descriptors to Block. example

Examples

[collapse all](#)

▼ Extract Corner Features From an Image.

Read the image.

Try This Example

Copy Command 

```
I = imread("cameraman.tif");
```

Find and extract corner features from the image.

```
corners = detectHarrisFeatures(I);
[features,valid_corners] = extractFeatures(I,corners);
```

Display image.

```
figure;
imshow(I);
hold on
```

Plot valid corner points.

```
plot(valid_corners);
```



▼ Extract SURF Features From an Image

Read image.

Try This Example

Copy Command 

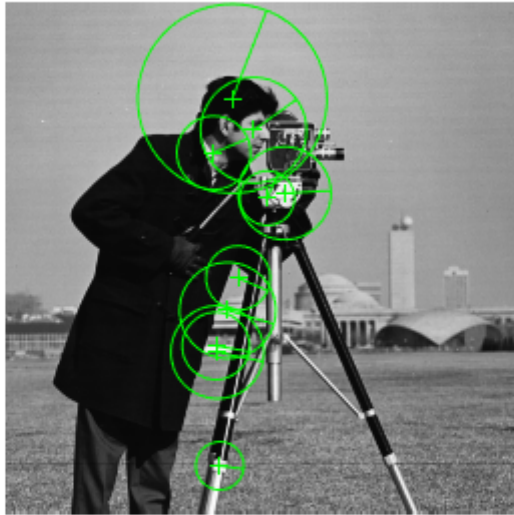
```
I = imread("cameraman.tif");
```

Find and extract features from the input image.

```
points = detectSURFFeatures(I);  
[features,valid_points] = extractFeatures(I,points);
```

Display and plot ten strongest SURF features.

```
figure;  
imshow(I);  
hold on;  
plot(valid_points.selectStrongest(10),showOrientation=true);
```



▼ Extract MSER Features From an Image

Read image.

Try This Example

Copy Command 

```
I = imread("cameraman.tif");
```

Find features in the image using MSER feature detection.

```
regions = detectMSERFeatures(I);  
[features,valid_points] = extractFeatures(I,regions,Upright=true);
```

Display features corresponding to the MSER ellipse centers.

```
figure;  
imshow(I);  
hold on;  
plot(valid_points,showOrientation=true);
```



Input Arguments

[collapse all](#)

- ✓ **I — Input image**
binary image | M -by- N 2-D grayscale image

Input image, specified as either a binary or 2-D grayscale image.

Data Types: logical | single | double | int16 | uint8 | uint16

- ✓ **points — Center location point**
 M -by-2 matrix | point feature object

Center location point of a square neighborhood, specified as an M -by-2 matrix of M number of $[x \ y]$ coordinates, or as one of the point feature objects described in Point Feature Types. The table lists the possible input classes of points that can be used for extraction.

Name-Value Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `extractFeatures(I,points,Method="Block")` sets the method to extract descriptors to `Block`.

- ✓ **Method — Descriptor extraction method**
"Auto" | point feature object | M -by-2 matrix

Descriptor extraction method, specified as one of the point feature objects described in Point Feature Types or "Auto".

The table describes how the function implements the descriptor extraction methods:

Method	Feature Vector (Descriptor)
SIFT	Scale-Invariant Feature Transform (SIFT). The function sets the Orientation property of the validPoints output object to the orientation of the extracted features, in radians.
BRISK	Binary Robust Invariant Scalable Keypoints (BRISK). The function sets the Orientation property of the validPoints output object to the orientation of the extracted features, in radians.
FREAK	Fast Retina Keypoint (FREAK). The function sets the Orientation property of the validPoints output object to the orientation of the extracted features, in radians.
SURF	Speeded-Up Robust Features (SURF). The function sets the Orientation property of the validPoints output object to the orientation of the extracted features, in radians. When you use an MSERRegions object with the SURF method, the Centroid property of the object extracts SURF descriptors. The Axes property of the object selects the scale of the SURF descriptors such that the circle representing the feature has an area proportional to the MSER ellipse area. The scale is calculated as $1/4 * \sqrt{(\text{majorAxes}/2) * (\text{minorAxes}/2)}$ and saturated to 1.6, as required by the SURFPoints object.
ORB	Oriented FAST and rotated BRIEF (ORB) features. The Orientation property of the validPoints output object is automatically set to the Orientation property of the input ORBPoints object points.
KAZE	Non-linear pyramid-based features. The function sets the Orientation property of the validPoints output object to the orientation of the extracted features, in radians. When you use an MSERRegions object with the KAZE method, the Location property of the object is used to extract KAZE descriptors. The Axes property of the object selects the scale of the KAZE descriptors such that the circle representing the feature has an area proportional to the MSER ellipse area.
Block	Simple square neighborhood. The Block method extracts only the neighborhoods fully contained within the image boundary. Therefore, the output, validPoints, can contain fewer points than the input POINTS.

Method	Feature Vector (Descriptor)				
Auto	The function selects the Method, based on the class of the input points and implements: <table><tr><td>The FREAK method for a cornerPoints input object.</td></tr><tr><td>The SURF method for a SURFPoints or MSERRegions input object.</td></tr><tr><td>The BRISK method for a BRISKPoints input object.</td></tr><tr><td>The ORB method for a ORBPoints input object.</td></tr></table> For an M-by-2 input matrix of $[x\ y]$ coordinates, the function implements the Block method.	The FREAK method for a cornerPoints input object.	The SURF method for a SURFPoints or MSERRegions input object.	The BRISK method for a BRISKPoints input object.	The ORB method for a ORBPoints input object.
The FREAK method for a cornerPoints input object.					
The SURF method for a SURFPoints or MSERRegions input object.					
The BRISK method for a BRISKPoints input object.					
The ORB method for a ORBPoints input object.					



Note

The descriptor extraction method must be ORB, if the input points is an ORBPoints object. Also, ORB descriptor extraction method is not supported for any other class of points, except ORBPoints.



BlockSize — Block size

11 (default) | odd integer scalar

Block size, specified as an odd integer scalar. This value defines the local square neighborhood BlockSize-by-BlockSize, centered at each interest point. This option applies only when the function implements the Block method.



Upright — Rotation invariance flag

false | logical scalar

Rotation invariance flag, specified a logical scalar. When you set this property to true, the orientation of the feature vectors are not estimated and the feature orientation is set to $\pi/2$. Set this to true when you do not need the image descriptors to capture rotation information. When you set this property to false, the orientation of the features is estimated and the features are then invariant to rotation.



Note

The rotation invariance flag "Upright" is not supported if the input points is an ORBPoints object.



FeatureSize — Length of feature vector

64 (default) | 128

Length of the feature vector (descriptor), specified as 64 or 128. This option applies only when the function implements the SURF or KAZE method. The larger feature size of 128 provides greater accuracy, but decreases the feature matching speed.

**features — Feature vectors***M*-by-*N* matrix | `binaryFeatures` object

Feature vectors, returned as a `binaryFeatures` object or an *M*-by-*N* matrix of *M* feature vectors, also known as descriptors. Each descriptor is of length *N*.

**validPoints — Valid points***M*-by-2 matrix of [x,y] coordinates | point feature object

Valid points associated with each output feature vector (descriptor) in `features`, returned in the same format as the input. Valid points can be a or an *M*-by-2 matrix of [x,y] coordinates or as one of the point feature objects described in Point Feature Types.

The function extracts descriptors from a region around each interest point. If the region lies outside of the image, or if interest point lies too close to the edge of the image, the function cannot compute the feature descriptor. In this case, the function ignores the point. The point is not included in the valid points output.

For SIFT features, more than one descriptor can be extracted at a single location, but at different orientations. Therefore, the number of points returned in `validPoints` can be greater than or equal to the number of input points to `extractFeatures`. For input feature points other than SIFT, `validPoints` can be less than or equal to the number of input points.

References

- [1] G. Bradski and A. Kaehler, *Learning OpenCV : Computer Vision with the OpenCV Library*, O'Reilly, Sebastopol, CA, 2008.
- [2] Herbert Bay, Andreas Ess, Tinne Tuytelaars, Luc Van Gool, *SURF: Speeded Up Robust Features*", *Computer Vision and Image Understanding (CVIU)*, Vol. 110, No. 3, pp. 346–359, 2008
- [3] Bay, Herbert, Andreas Ess, Tinne Tuytelaars, and Luc Van Gool, "SURF: Speeded Up Robust Features", *Computer Vision and Image Understanding (CVIU)*, Vol. 110, No. 3, pp. 346–359, 2008.
- [4] Alahi, Alexandre, Ortiz, Raphael, and Pierre Vandergheynst, "FREAK: Fast Retina Keypoint", *IEEE Conference on Computer Vision and Pattern Recognition*, 2012.
- [5] Alcantarilla, P.F., A. Bartoli, and A.J. Davison. "KAZE Features", *ECCV 2012, Part VI, LNCS 7577* pp. 214, 2012

Extended Capabilities

> C/C++ Code Generation

Generate C and C++ code using MATLAB® Coder™.

Version History

Introduced in R2011a

See Also

Topics

[Point Feature Types](#)

[Local Feature Detection and Extraction](#)